

Project Report
ABC Foodmart Database System

Group 2

*Yike Meng (ym3137) | Olivia Chou (cc5399) | Maggie Teng (mt3952) | Afeefah Moazzam
(am6976)*

MS Applied Analytics, Columbia University

APANPS5310: SQL

Instructor: Prof. Nikolaos Machairas

April 28, 2026

Table of Contents

1. Problem Statement.....	3
2. Proposal.....	4
3. Team Structure and Timeline	5
Team Members and Responsibilities	5
Timeline of Activities.....	5
4. Database Schema.....	6
Design Overview.....	6
Entity-Relationship Diagram.....	6
Key Design Decisions.....	7
SQL Schema (Selected Tables).....	7
5. ETL Process.....	8
Overview	8
Insertion Order and Referential Integrity	9
Data Volumes.....	9
Key ETL Code	10
Query 1 – Total Revenue by Store.....	10
Query 2 – Product Revenue Ranking (Window Function).....	11
Query 3 – Low Inventory Alert	11
Query 4 – Labor Cost by Store and Month.....	11
Query 5 – Top Customers by Loyalty and Spend	12
Query 6 – Daily Revenue Trend per Store.....	12
Query 7 – Vendor Delivery Performance	12
Query 8 – Employee Overtime Analysis.....	13
Query 9 – Category Revenue Share per Store	13
Query 10 – Month-over-Month Revenue Growth	13
User Types	14
Analyst Interaction.....	14
Executive Interaction – Metabase Dashboard.....	14
Dashboard Screenshots	15
Performance and Redundancy.....	16
9. Appendix: Business Requirements	18

1. Problem Statement

ABC Foodmart is a neighborhood grocery chain with two established locations in Queens, New York, that has operated for over 30 years. Under new management, the company is planning a significant expansion: three new store locations in Brooklyn, bringing the total to five stores across two boroughs. This expansion represents a critical inflection point for the business.

Prior to this project, ABC Foodmart managed all operational data — including staffing records, inventory levels, vendor relationships, delivery schedules, sales transactions, and financial accounts — through a combination of spreadsheets and paper binders. This approach functioned adequately when managing two locations, but is fundamentally unsuitable for a multi-location operation. Key problems included:

- No centralized view of inventory across locations, making restocking decisions slow and error-prone.
- Sales and revenue data stored in disconnected spreadsheets, making cross-store financial comparisons manual and time-consuming.
- No systematic tracking of customer purchasing behavior or loyalty program data.
- Vendor and delivery records maintained on paper, creating risk of data loss and making supplier performance analysis impossible.
- Payroll calculations performed manually, with no integration with scheduling or hours-worked data.

Management recognized that expanding to five locations with these systems in place would lead to operational failures, financial errors, and an inability to make informed decisions in real time. Our team was engaged to design and implement a centralized relational database system that would replace the existing spreadsheet infrastructure, automate key data flows, and provide both operational staff and executive leadership with the information they need.

Our client has asked that the database meets the following core requirements:

- Store and manage data on all store locations, employees, departments, and work schedules.
- Track all vendors, products, deliveries, and inventory levels with automated reorder alerts.
- Record all sales transactions with line-item detail, customer loyalty tracking, and promotional discount support.
- Capture all store operating expenses by category and calculate payroll from hourly wage records.
- Calculate and compare the profitability and performance of each store location.

- Provide real-time dashboards for management and support direct SQL querying for analysts.

2. Proposal

Our team proposed the design and implementation of a fully normalized relational database system built on PostgreSQL, replacing all existing spreadsheet and paper-based records. The system is organized around four functional modules that mirror ABC Foodmart's operational structure: Store & Staff, Supply Chain & Inventory, Sales & Customers, and Finance.

The database consists of 16 tables in Third Normal Form (3NF), ensuring data integrity, eliminating redundancy, and preventing update anomalies. Three automated triggers handle key business logic without requiring manual intervention: inventory levels are automatically decremented after each sale, customer loyalty points are credited after each transaction, and inventory timestamps are updated whenever stock levels change.

For executive leadership, we proposed and implemented an interactive dashboard in Metabase, a business intelligence platform connected directly to the PostgreSQL database. This dashboard provides real-time visibility into store revenue, labor costs, top-selling products, inventory alerts, and customer behavior — all without requiring any SQL knowledge from the viewer.

For data analysts, the system supports direct SQL querying via psql or DBeaver, with a suite of pre-designed analytical queries covering profitability, product performance, staffing costs, customer loyalty, and more.

The primary benefits of this system to ABC Foodmart include:

- Faster, more accurate decisions through real-time data access across all five store locations.
- Elimination of manual data entry errors through referential integrity constraints and automated triggers.
- Scalability: the schema is designed to accommodate additional stores, product categories, and vendors without structural changes.
- A foundation for future analytics and potential customer-facing applications such as a loyalty app.

3. Team Structure and Timeline

Team Members and Responsibilities

Name	Role	Responsibilities
Afeefah Moazzam (am6976)	Business Analyst	Business requirements, planning, problem statement, proposal, and final report coordination.
Yike Meng (ym3137)	Database Architect	Entity-relationship diagram, full CREATE TABLE SQL, constraints, triggers, and profitability view.
Maggie Teng (mt3952)	Data Engineer	Python data population script, ETL plan, sample data generation, and referential integrity testing.
Olivia Chou (cc5399)	Analytics & Reporting Lead	SQL analytical queries, Metabase dashboard setup, customer interaction plan, and presentation.

Timeline of Activities

Period	Activity
March 30 – April 1	Finalized scenario selection, business requirements, and team contract (Checkpoint 2).
April 3 – April 8	Designed full database schema, wrote CREATE TABLE SQL, resolved circular FK, added triggers and view (Checkpoint 3).
April 9 – April 15	Developed Python data population script using faker and psycopg2; populated all 16 tables (Checkpoint 4).
April 16 – April 22	Wrote analytical SQL queries, set up Metabase in Docker, built executive dashboard (Checkpoint 5).
April 23 – April 27	Final report writing, dashboard refinement, and presentation preparation.
April 28	Final presentation and report submission.

The most challenging tasks were: (1) resolving the circular foreign key dependency between Stores and Employees (the manager_id field), which required a two-step approach of creating Stores without the FK constraint and adding it via ALTER TABLE after Employees was populated; and (2) managing Docker network configuration to allow Metabase to connect to the PostgreSQL container via internal IP address rather than localhost.

4. Database Schema

Design Overview

The database consists of 16 tables organized into four modules. All tables are in Third Normal Form: every non-key attribute depends on the whole primary key and nothing but the primary key. Foreign key constraints enforce referential integrity, and CHECK constraints ensure data validity at the column level.

Module	Tables	Description
Store & Staff	Stores, Departments, Employees, Schedules	Manages all location, personnel, and scheduling data.
Supply Chain & Inventory	Vendors, Products, Inventory, Deliveries, Delivery_Items	Tracks vendor relationships, product catalog, stock levels, and incoming deliveries.
Sales & Customers	Customers, Transactions, Transaction_Items, Promotions	Records all sales activity, loyalty program, and promotional discounts.
Finance	Expense_Categories, Expenses, Payroll	Captures operating costs and hourly payroll by employee and store.

Entity-Relationship Diagram

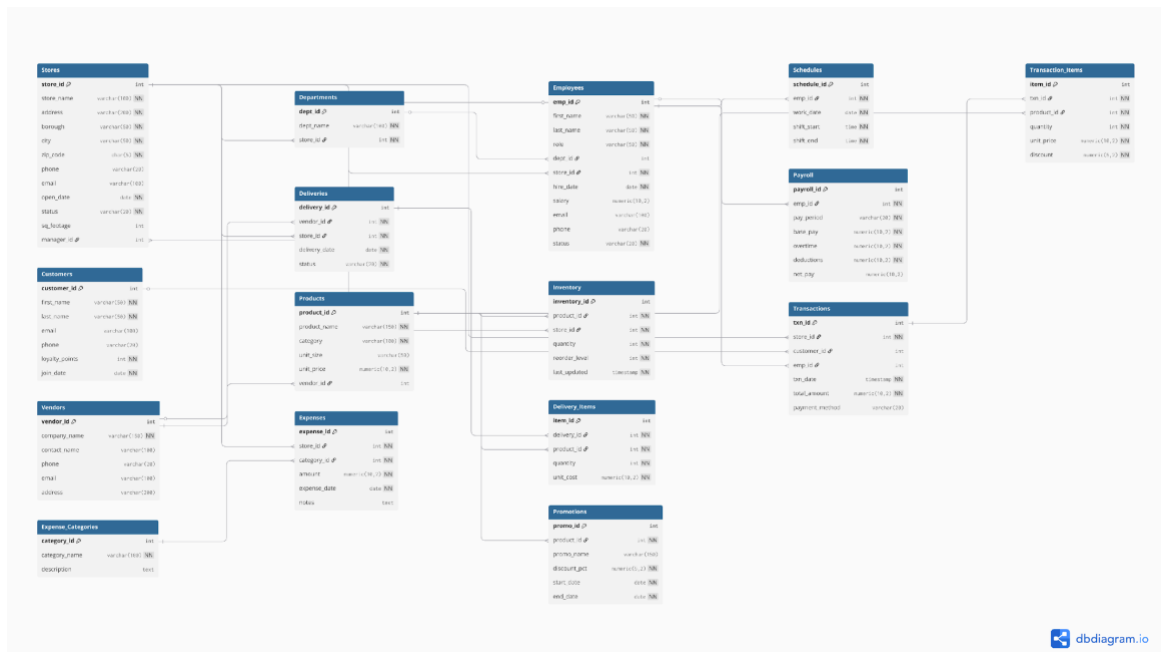


Figure 1: Entity-Relationship Diagram – ABC Foodmart Database (16 tables)

Key Design Decisions

- Circular FK resolution: Stores.manager_id references Employees, but Employees.store_id references Stores. We create Stores first without the manager FK, create Employees, then add the constraint via ALTER TABLE to avoid a dependency deadlock.
- Hourly wage model: Employees store an hourly_wage rather than an annual salary. Payroll records capture hours_worked and overtime_hours, with base_pay and overtime calculated from the wage rate. net_pay is a GENERATED ALWAYS AS computed column.
- Guest checkout: Transactions.customer_id is nullable, allowing walk-in purchases to be recorded without requiring a customer profile.
- Fixed pricing: Products.unit_price is the authoritative retail price. Transaction_Items.unit_price is populated from this value at time of sale, preserving a historical record even if the product price changes later. Delivery_Items.unit_cost is set at 50–70% of the retail price, reflecting realistic wholesale margins.
- Inventory triggers: Three triggers automate key operations — decrementing inventory after each sale, crediting loyalty points after each transaction, and updating the inventory timestamp on any stock change.

SQL Schema (Selected Tables)

The complete SQL schema is provided in the accompanying file abc_foodmart_schema.sql. Key table definitions are shown below.

```
-- Stores (top-level entity; manager_id FK added after Employees)
CREATE TABLE Stores (
  store_id      SERIAL          PRIMARY KEY,
  store_name    VARCHAR(100)    NOT NULL,
  address       VARCHAR(200)    NOT NULL,
  borough       VARCHAR(50)     NOT NULL CHECK (borough IN ('Queens', 'Brooklyn')),
  city          VARCHAR(50)     NOT NULL DEFAULT 'New York',
  zip_code      CHAR(5)         NOT NULL,
  phone         VARCHAR(20)     UNIQUE,
  open_date     DATE            NOT NULL,
  status        VARCHAR(20)     NOT NULL DEFAULT 'active'
                                CHECK (status IN
('active', 'closed', 'under_renovation')),
  sq_footage    INTEGER         CHECK (sq_footage > 0),
  manager_id    INTEGER         -- FK added via ALTER TABLE
);
```

```
-- Payroll (net_pay is a computed column)
CREATE TABLE Payroll (
    payroll_id    SERIAL          PRIMARY KEY,
    emp_id        INTEGER         NOT NULL REFERENCES Employees(emp_id),
    pay_period    VARCHAR(20)     NOT NULL,
    hours_worked  NUMERIC(6,2)    NOT NULL CHECK (hours_worked >= 0),
    overtime_hours NUMERIC(6,2)    NOT NULL DEFAULT 0.00,
    base_pay      NUMERIC(10,2)   NOT NULL CHECK (base_pay >= 0),
    overtime      NUMERIC(10,2)   NOT NULL DEFAULT 0.00,
    deductions    NUMERIC(10,2)   NOT NULL DEFAULT 0.00,
    net_pay       NUMERIC(10,2)   GENERATED ALWAYS AS
        (base_pay + overtime - deductions) STORED,
    UNIQUE (emp_id, pay_period)
);
```

```
-- Trigger: reduce inventory after sale
CREATE OR REPLACE FUNCTION reduce_inventory_on_sale()
RETURNS TRIGGER AS $$
DECLARE v_store_id INTEGER;
BEGIN
    SELECT store_id INTO v_store_id FROM Transactions WHERE txn_id = NEW.txn_id;
    UPDATE Inventory SET quantity = quantity - NEW.quantity
    WHERE product_id = NEW.product_id AND store_id = v_store_id;
    RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_reduce_inventory
AFTER INSERT ON Transaction_Items
FOR EACH ROW EXECUTE FUNCTION reduce_inventory_on_sale();
```

```
-- Profitability view
CREATE OR REPLACE VIEW store_profitability AS
SELECT s.store_id, s.store_name, s.borough,
    COALESCE(SUM(t.total_amount), 0) AS total_revenue,
    COALESCE(SUM(e.amount), 0) AS total_expenses,
    COALESCE(SUM(p.base_pay + p.overtime), 0) AS total_payroll
FROM Stores s
LEFT JOIN Transactions t ON t.store_id = s.store_id
LEFT JOIN Expenses e ON e.store_id = s.store_id
LEFT JOIN Employees emp ON emp.store_id = s.store_id
LEFT JOIN Payroll p ON p.emp_id = emp.emp_id
GROUP BY s.store_id, s.store_name, s.borough;
```

5. ETL Process

Overview

Sample data was generated and loaded using a Python script (populate_data.py) built with the faker library for realistic synthetic data and psycopg2 for direct PostgreSQL connectivity. A

fixed random seed (`random.seed(42)`) ensures results are reproducible. All inserts are wrapped in a single transaction with rollback on error.

Insertion Order and Referential Integrity

Data is inserted strictly in foreign key dependency order to maintain referential integrity throughout the loading process:

Step	Table(s)	Key Design Decision
1	Stores	Inserted without <code>manager_id</code> FK to avoid circular dependency.
2–3	Departments, Employees	Employees assigned to departments and stores.
4	Stores (UPDATE)	<code>manager_id</code> set via UPDATE after employees exist.
5	Schedules	~12 shifts per employee generated.
6–9	Vendors, Products, Inventory, Deliveries, Delivery_Items	Unit costs set at 50–70% of retail price.
10–12	Customers, Transactions, Transaction_Items	Inventory checked before each insert to prevent negative stock.
13–15	Promotions, Expense_Categories, Expenses, Payroll	Payroll computed from <code>hourly_wage × hours_worked</code> .

Data Volumes

Table	Rows	Notes
Stores	5	2 Queens, 3 Brooklyn
Departments	20	4 per store
Employees	50	10 per store, role-based hourly wages
Schedules	600	~12 shifts per employee
Vendors	8	Named NYC-area suppliers
Products	50	5 categories, fixed retail prices
Inventory	250	1 record per product per store
Deliveries + Items	30 + 147	~6 deliveries per store

Table	Rows	Notes
Customers	200	Mix of loyalty members and guests
Transactions + Items	1,000 + 3,726	200 per store, Jan–Apr 2025
Promotions	20	Random product discounts
Expenses	150	10 per store per month, 3 months
Payroll	150	1 record per employee per month

Key ETL Code

```
# Payroll: compute from hourly wage
cur.execute('SELECT hourly_wage FROM Employees WHERE emp_id = %s', (emp_id,))
hourly_wage = float(cur.fetchone()[0])
hours_worked = round(random.uniform(120, 168), 2)
overtime_hours = round(random.uniform(0, 15), 2)
base_pay = round(hours_worked * hourly_wage, 2)
overtime = round(overtime_hours * hourly_wage * 1.5, 2)
```

```
# Transaction items: check inventory before inserting
cur.execute('SELECT quantity FROM Inventory WHERE product_id=%s AND store_id=%s',
(pid, sid))
row = cur.fetchone()
if row is None or row[0] <= 0:
    continue
qty = min(random.randint(1, 5), row[0]) # never exceed available stock
```

6. Analytics Applications

The following ten analytical procedures were designed to address specific business questions from ABC Foodmart’s management. Each query goes beyond simple counts or averages to deliver actionable insights. All queries were tested against the populated database.

Query 1 – Total Revenue by Store

Business question: Which stores are generating the most revenue? This allows management to identify high-performing locations and benchmark new Brooklyn stores against established Queens locations.

```
SELECT store_name, borough, total_revenue
FROM store_profitability
ORDER BY total_revenue DESC;
```

Query 2 – Product Revenue Ranking (Window Function)

Business question: How do all products rank by revenue? Using RANK() instead of LIMIT makes the query comprehensive — analysts can filter by rank threshold rather than being constrained to a fixed top-N cutoff.

```
SELECT product_name, category, revenue,
       RANK() OVER (ORDER BY revenue DESC) AS revenue_rank
FROM (
  SELECT p.product_name, p.category,
         SUM(ti.quantity * ti.unit_price * (1 - ti.discount/100)) AS revenue
  FROM   Transaction_Items ti
  JOIN   Products p ON p.product_id = ti.product_id
  JOIN   Transactions t ON t.txn_id = ti.txn_id
  WHERE  t.txn_date >= '2025-01-01'
  GROUP BY p.product_name, p.category
) ranked_products
ORDER BY revenue_rank;
```

Query 3 – Low Inventory Alert

Business question: Which products need to be reordered at each store? This query directly supports operations staff in prioritizing restocking decisions before shelves run empty.

```
SELECT s.store_name, p.product_name, p.category,
       i.quantity, i.reorder_level,
       (i.reorder_level - i.quantity) AS units_below_threshold
FROM   Inventory i
JOIN   Stores s   ON s.store_id   = i.store_id
JOIN   Products p ON p.product_id = i.product_id
WHERE  i.quantity <= i.reorder_level
ORDER BY s.store_name, (i.reorder_level - i.quantity) DESC;
```

Query 4 – Labor Cost by Store and Month

Business question: How does labor cost vary by store and over time? This query enables the CFO to identify stores with disproportionately high payroll relative to revenue and investigate staffing efficiency.

```
SELECT s.store_name, s.borough, pr.pay_period,
       SUM(pr.base_pay + pr.overtime) AS total_labor_cost,
       SUM(pr.overtime)              AS total_overtime
FROM   Payroll pr
JOIN   Employees e ON e.emp_id   = pr.emp_id
JOIN   Stores s   ON s.store_id = e.store_id
GROUP BY s.store_name, s.borough, pr.pay_period
ORDER BY pr.pay_period, total_labor_cost DESC;
```

Query 5 – Top Customers by Loyalty and Spend

Business question: Who are our most valuable customers? This ranking combines total spend with loyalty points and transaction frequency to identify customers most suitable for targeted retention campaigns.

```
SELECT c.first_name || ' ' || c.last_name AS customer_name,
       c.loyalty_points,
       COUNT(t.txn_id) AS total_transactions,
       SUM(t.total_amount) AS total_spend,
       ROUND(AVG(t.total_amount)::numeric, 2) AS avg_basket_size
FROM   Customers c
JOIN   Transactions t ON t.customer_id = c.customer_id
GROUP  BY c.customer_id, c.first_name, c.last_name, c.loyalty_points
ORDER  BY total_spend DESC
LIMIT 20;
```

Query 6 – Daily Revenue Trend per Store

Business question: Are there patterns in daily revenue that could inform staffing and inventory planning? This query surfaces day-level revenue fluctuations across all five stores.

```
SELECT s.store_name,
       DATE(t.txn_date) AS sale_date,
       COUNT(t.txn_id) AS num_transactions,
       SUM(t.total_amount) AS daily_revenue
FROM   Transactions t
JOIN   Stores s ON s.store_id = t.store_id
GROUP  BY s.store_name, DATE(t.txn_date)
ORDER  BY sale_date, s.store_name;
```

Query 7 – Vendor Delivery Performance

Business question: Which vendors are most reliable? This query computes each vendor's delivery completion rate, helping procurement managers renegotiate contracts or switch suppliers.

```
SELECT v.company_name,
       COUNT(*) AS total_deliveries,
       SUM(CASE WHEN d.status='delivered' THEN 1 ELSE 0 END) AS completed,
       ROUND(100.0 * SUM(CASE WHEN d.status='delivered'
                          THEN 1 ELSE 0 END) / COUNT(*), 1) AS completion_rate_pct
FROM   Deliveries d
JOIN   Vendors v ON v.vendor_id = d.vendor_id
GROUP  BY v.company_name
ORDER  BY completion_rate_pct DESC;
```

Query 8 – Employee Overtime Analysis

Business question: Which employees are consistently working overtime, and at which stores?
Identifying overtime patterns allows managers to optimize scheduling and control labor costs.

```
SELECT e.first_name || ' ' || e.last_name AS employee_name,
       e.role, s.store_name,
       SUM(pr.overtime_hours)           AS total_overtime_hours,
       SUM(pr.overtime)                 AS total_overtime_pay
FROM   Payroll pr
JOIN   Employees e ON e.emp_id = pr.emp_id
JOIN   Stores s     ON s.store_id = e.store_id
GROUP BY e.emp_id, e.first_name, e.last_name, e.role, s.store_name
HAVING SUM(pr.overtime_hours) > 10
ORDER BY total_overtime_hours DESC;
```

Query 9 – Category Revenue Share per Store

Business question: What proportion of each store’s revenue comes from each product category?
This helps buyers understand local demand patterns and tailor inventory to each borough’s preferences.

```
SELECT s.store_name, p.category,
       SUM(ti.quantity * ti.unit_price * (1 - ti.discount/100)) AS
category_revenue,
       ROUND(100.0 * SUM(ti.quantity * ti.unit_price * (1 - ti.discount/100))
             / SUM(SUM(ti.quantity * ti.unit_price * (1 - ti.discount/100))
                   OVER (PARTITION BY s.store_name), 2))
       AS
pct_of_store_revenue
FROM   Transaction_Items ti
JOIN   Transactions t     ON t.txn_id = ti.txn_id
JOIN   Stores s          ON s.store_id = t.store_id
JOIN   Products p        ON p.product_id = ti.product_id
GROUP BY s.store_name, p.category
ORDER BY s.store_name, category_revenue DESC;
```

Query 10 – Month-over-Month Revenue Growth

Business question: Is revenue growing or declining month over month? Using LAG() to compare each month’s revenue against the prior month gives management a clear view of growth trajectory across the store network.

```
WITH monthly AS (
  SELECT s.store_name,
         DATE_TRUNC('month', t.txn_date) AS month,
         SUM(t.total_amount)           AS monthly_revenue
  FROM   Transactions t
  JOIN   Stores s ON s.store_id = t.store_id
  GROUP BY s.store_name, DATE_TRUNC('month', t.txn_date)
```

```

)
SELECT store_name, month, monthly_revenue,
       LAG(monthly_revenue) OVER
         (PARTITION BY store_name ORDER BY month) AS prior_month_revenue,
       ROUND(100.0 * (monthly_revenue -
         LAG(monthly_revenue) OVER (PARTITION BY store_name ORDER BY month))
         / NULLIF(LAG(monthly_revenue) OVER
         (PARTITION BY store_name ORDER BY month), 0), 2) AS mom_growth_pct
FROM   monthly
ORDER  BY store_name, month;

```

7. Customer Interaction Plan

User Types

User Type	Technical Level	Primary Interaction Method
Data Analyst	High – writes SQL	Direct querying via psql or DBeaver connected to PostgreSQL
Store Manager	Medium	Metabase dashboards filtered by their store
C-Level Officer	Low – no SQL	Metabase executive dashboard with cross-store KPIs

Analyst Interaction

Analysts connect directly to the PostgreSQL instance using psql or DBeaver with read-only credentials. The 10 analytical procedures above represent the core query suite. Python scripts using psycopg2 can also be used to automate report generation or schedule recurring analyses. The benefit of SQL over spreadsheets is the ability to join across multiple tables, apply window functions, and compute derived metrics that would require manual multi-step processes in Excel.

Executive Interaction – Metabase Dashboard

C-level officers access a live Metabase dashboard connected to the PostgreSQL database. The dashboard requires no SQL knowledge and is organized into three tabs:

Tab	Content
Financial Overview	Total Revenue by Store (bar chart), Labor Cost by Store (bar chart)
Sales & Products	Daily Revenue Trend (line), Top 10 Products by Revenue (horizontal bar), Daily Transaction Volume (multi-line)
Operations	Inventory Alert table (75 low-stock items), Top 20 Customers by Spend table

Dashboard Screenshots



Figure 2: Financial Overview Tab – Total Revenue and Labor Cost by Store

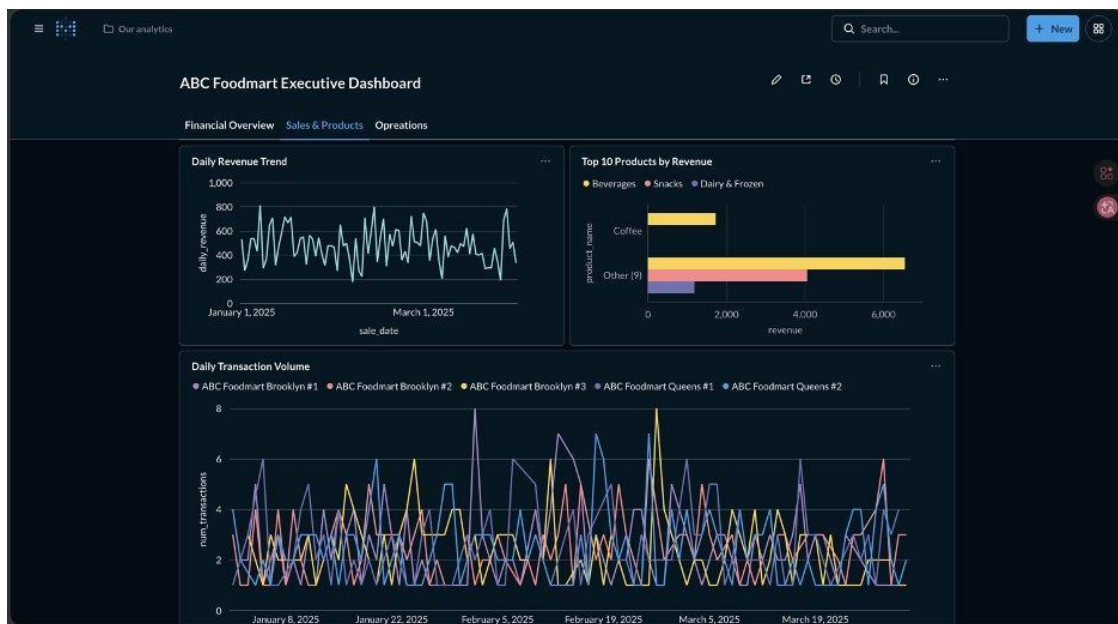


Figure 3: Sales & Products Tab – Revenue Trend, Top Products, Transaction Volume

store_name	product_name	category	quantity
ABC Foodmart Brooklyn #1	Strawberries	Produce	0
ABC Foodmart Brooklyn #1	Mixed Nuts	Snacks	0
ABC Foodmart Brooklyn #1	Bagels	Bakery	0
ABC Foodmart Brooklyn #1	Avocados	Produce	0
ABC Foodmart Brooklyn #1	2% Milk	Dairy & Frozen	0
ABC Foodmart Brooklyn #1	Whole Milk	Dairy & Frozen	0
ABC Foodmart Brooklyn #1	Muffins	Bakery	0
ABC Foodmart Brooklyn #1	Croissants	Bakery	0

customer_name	loyalty_points	total_transactions	total_spend
Edward Brown	1,719	13	640.78
Wayne Wolfe	1,628	12	601.04
Daniel Torres	64	11	526.8
Anna Lopez	1,955	11	496.13
Brandon Cooper	1,380	10	459.34
Vanessa Phillips	25	12	423.03
James Johnson	1,392	10	422.08
Timothy Booth	1,880	7	414.48

Figure 4: Operations Tab – Inventory Alert and Top Customer Rankings

Performance and Redundancy

Performance: The `store_profitability` view pre-aggregates key financial metrics. Indexes are planned on `Transactions.store_id`, `Transactions.txn_date`, `Transaction_Items.txn_id`, `Inventory.store_id`, and `Payroll.emp_id` — the columns most frequently used in `WHERE` and `JOIN` clauses. Metabase caches results on a 24-hour refresh cycle.

Redundancy: The system is designed around a primary-standby PostgreSQL architecture. A standby server receives continuous updates from the primary via streaming replication and serves as an automatic failover target if the primary becomes unavailable. This ensures service continuity for both analysts and the Metabase dashboard. Periodic `pg_dump` backups provide an additional point-in-time recovery option.

Hosting: For a production deployment, we recommend cloud hosting (AWS RDS or Google Cloud SQL) for managed replication, automated backups, and scalability — eliminating the need for in-house database administration infrastructure.

8. Conclusion

This project delivered a complete relational database system for ABC Foodmart, replacing a fragmented collection of spreadsheets and paper records with a centralized, normalized, and automated data infrastructure. The system consists of 16 tables in 3NF, three automated triggers, a profitability view, a full ETL pipeline, ten analytical SQL procedures, and an interactive executive dashboard in Metabase.

The primary benefits realized for ABC Foodmart are:

- Real-time visibility across all five store locations through a single database system, enabling management to make faster and more informed decisions.
- Elimination of manual data entry errors through foreign key constraints, CHECK constraints, and automated triggers that handle inventory, loyalty points, and payroll calculations without human intervention.
- Analytical capabilities that were impossible with spreadsheets: cross-store revenue comparisons, month-over-month growth tracking, category revenue share analysis, vendor reliability scoring, and employee overtime monitoring.
- A scalable foundation: the schema accommodates additional stores, product lines, and vendors without structural changes, positioning ABC Foodmart for continued growth beyond the initial Brooklyn expansion.
- Executive accessibility: C-level officers can now access live operational metrics through an interactive dashboard without requiring SQL knowledge or reliance on analyst-prepared reports.

The transition from unstructured data to a relational database management system represents a fundamental upgrade in ABC Foodmart's operational capabilities. The insights made possible by this system — from identifying which stores drive the most revenue to which products are at risk of stockout — directly support the company's ability to manage its expanded five-store network efficiently and profitably.

9. Appendix: Business Requirements

#	Business Requirement	Module
1	Store and manage data on all store locations, including address, borough, opening date, and operational status	Store & Staff
2	Track all employees across locations, including their role, department, hire date, and hourly wage	Store & Staff
3	Manage employee work schedules, including shift times and assigned dates	Store & Staff
4	Organize employees into departments within each store	Store & Staff
5	Maintain a master product catalog with fixed retail prices	Supply Chain
6	Store and manage vendor data including contact information	Supply Chain
7	Track all incoming deliveries with delivery date and status	Supply Chain
8	Monitor inventory levels per product per store with reorder thresholds	Supply Chain
9	Record all customer sales transactions with line-item detail	Sales
10	Track individual transaction line items including discounts applied	Sales
11	Manage customer profiles and a loyalty points program	Sales
12	Support time-limited promotional discounts on products	Sales
13	Record and categorize all store operating expenses	Finance
14	Track employee payroll from hourly wage and hours worked	Finance
15	Calculate store profitability by comparing revenue against costs	Finance
16	Compare financial performance across store locations	Finance
17	Provide real-time executive dashboards showing key metrics	Analytics
18	Enable complex SQL queries for operational analysis	Analytics
19	Support historical data retention for trend analysis	Analytics